National University of Computer & Emerging Sciences, Karachi
Computer Science Department
Spring 2026, Lab Manual – 09

Course Code: CL-1004	**Course : Object Oriented Programming Lab**
Instructor :	**Muhammad Owais**

LAB - 9

Polymorphism & its types

Polymorphism:

Polymorphism is a concept in Object-Oriented Programming (OOP) that means “one thing can have many forms. It means the same method name can perform different tasks depending on how it is used or which object calls it. This allows programmers to reuse the same method name while changing its behavior according to the situation. Polymorphism makes the code more flexible, easier to maintain, and more organized because one interface can represent multiple behaviors.

Example:

Think about a person speaking:

- When speaking to a friend, the tone is casual.
- When speaking to a teacher, the tone is respectful.
- When speaking to a child, the tone is simple.

The action is the same (speaking) but the behavior changes depending on the situation. This is exactly how polymorphism works in programming.

Types of Polymorphism

Java mainly has two types of polymorphism:

1. Compile-Time Polymorphism (Method Overloading)
2. Run-Time Polymorphism (Method Overriding)

1. Compile-Time Polymorphism (Method Overloading):

Compile-time polymorphism occurs when multiple methods in the same class have the same name but different parameters, such as different number or type of arguments. The compiler decides which method should be executed during compilation based on the arguments passed.

Methods must have:

- Same method name
- Different Parameters (type or number)

```java
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {

        Calculator c = new Calculator();

        System.out.println(c.add(2,3));       // method 1
        System.out.println(c.add(2,3,4));     // method 2
        System.out.println(c.add(2.5,3.5));   // method 3
    }
}
```

Output:

```
5
9
6.0
```

Here the method name is the same (add) but:

- add(int,int)
- add(int,int,int)
- add(double,double)

Java decides which method to run based on parameters.

So this is Compile-Time Polymorphism.

2. Run-Time Polymorphism (Method Overriding)

Run-time polymorphism occurs when a child class provides its own implementation of a method that is already defined in the parent class. The method name and parameters remain the

same, but the method body changes. The decision about which method to execute is made when the program runs, depending on the object.

This is also called Dynamic Method Dispatch.

Rules

- Must use inheritance
- Method name must be same
- Parameters must be same
- Only method body changes

Example 1:

```
class Animal {

    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {

    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {

    void sound() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {

        Animal a;

        a = new Dog();
        a.sound();

        a = new Cat();
        a.sound();
    }
}
```

Output:

```
Dog barks
Cat meows
```

Parent class => Animal

- Child classes => Dog, Cat

All classes have sound() method.

But the behavior changes depending on the object.

Animal a = new Dog(); => Dog sound()
Animal a = new Cat(); => Cat sound()

Even though reference is Animal, the object decides the method. That is Runtime Polymorphism.

Example 2:

```
class Employee {

    String name;
    double baseSalary;

    Employee(String name, double baseSalary){
        this.name = name;
        this.baseSalary = baseSalary;
    }

    double calculateSalary(){
        return baseSalary;
    }
}

class Manager extends Employee{

    double bonus;

    Manager(String name, double baseSalary, double bonus){
        super(name, baseSalary);
        this.bonus = bonus;
    }

    double calculateSalary(){
        return baseSalary + bonus;
    }
}
```

```java
class Developer extends Employee{

    double overtime;

    Developer(String name, double baseSalary, double overtime){
        super(name, baseSalary);
        this.overtime = overtime;
    }

    double calculateSalary(){
        return baseSalary + overtime;
    }
}
 public class Main {

    public static void main(String[] args){

        Employee e;

        e = new Manager("Ali", 50000, 10000);
        System.out.println(e.calculateSalary());

        e = new Developer("Ahmed", 40000, 5000);
        System.out.println(e.calculateSalary());
    }
}
```

Output:

```
60000.0
45000.0
```

All classes use the same method calculateSalary(), but:

- Manager adds bonus
- Developer adds overtime

Even though the reference is Employee, the object decides which method runs at runtime.